

Q1.

1. Write the C program for calculation of *distance* and *angle*.

```
int main()
{
    int pos_x1 = 10;
    int pos_y1 = 20;
    int pos_x2 = 40;
    int pos_y2 = 60;
    int dx = pos_x2 - pos_x1;
    int dy = pos_y2 - pos_y1;

    int squared_sum = dx * dx + dy * dy;
    int distance = 0;

    while (1)
    { //(distance + 1) * (distance + 1) <=
squared_sum
        int dis2 = distance + 1;
        int dis2pow2 = dis2 * dis2;
        if(dis2pow2 <= squared_sum){
            distance = distance + 1;
        }
        else{
            break;
        }
    }
    int angle;
    if (dx == 0)
    {
        angle = 90;
    }
    else
    {
        int ratio = (dy * 100) / dx;

        if (ratio <= 8)
            angle = 5;
        else if (ratio <= 17)
```

```
            angle = 10;
        else if (ratio <= 27)
            angle = 15;
        else if (ratio <= 36)
            angle = 20;
        else if (ratio <= 47)
            angle = 25;
        else if (ratio <= 58)
            angle = 30;
        else if (ratio <= 70)
            angle = 35;
        else if (ratio <= 84)
            angle = 40;
        else if (ratio <= 100)
            angle = 45;
        else if (ratio <= 119)
            angle = 50;
        else if (ratio <= 143)
            angle = 55;
        else if (ratio <= 173)
            angle = 60;
        else if (ratio <= 214)
            angle = 65;
        else if (ratio <= 275)
            angle = 70;
        else if (ratio <= 373)
            angle = 75;
        else if (ratio <= 567)
            angle = 80;
        else
            angle = 85;
    }
    int distance_result = distance;
    int angle_result = angle;
}
```

2. Compile the C program you developed using SAYAC Compiler.
Compile:

```
C Drone.c x README.md
tests > CodeGeneration > Drone > C Drone.c > main()
1  int main()
2  {
3      int pos_x1 = 10;
4      int pos_y1 = 20;
5      int pos_x2 = 40;
6      int pos_y2 = 60;
7      int dx = pos_x2 - pos_x1;
8      int dy = pos_y2 - pos_y1;
9
10     int squared_sum = dx * dx + dy * dy;
11     int distance = 0;
12
13     while (1)
14     { //(distance + 1) * (distance + 1) <= squared_sum
15         int dis2 = distance + 1;
16         int dis2pow2 = dis2 * dis2;
17         if(dis2pow2 <= squared_sum){
18             distance = distance + 1;
19         }
20         else{
21             break;
22         }
23     }
24
25     int angle;
26
27     if (dx == 0)
28     {
29         angle = 90;
30     }
31     else
32     {
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● ali@:~/Downloads/SAYAC_Compiler-main$ java -cp "out:lib/antlr-4.13.1-complete.jar" SimpleLang tests/CodeGeneration/Drone/Drone.c -o out/output.s
● ali@:~/Downloads/SAYAC_Compiler-main$
```

Assemble:

```
C Drone.c  test1.in x
assembler > assembler_tests > test1.in
440 After_If 20:
441 After_If 23:
442 After_If 20:
443 After_If 17:
444 After_If 14:
445 After_If 11:
446 After_If 8:
447 ADI -16 FP
448 LDR FP R1
449 ADI 16 FP
450 ADR ZR R1 R2
451 ADI -18 FP
452 LDR FP R3
453 ADI 18 FP
454 ADR ZR R3 R4
455 ADI -16 FP
456 STR R1 FP
457 ADI 16 FP
458 ADI -20 FP
459 STR R2 FP
460 ADI 20 FP
461 ADI -18 FP
462 STR R3 FP
463 ADI 18 FP
464 ADI -22 FP
465 STR R4 FP
466 ADI 22 FP
467 func_ret_main:
468 ADR ZR FP SP
469 LDR FP FP

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
* ali@:~/Downloads/SAYAC_Compiler-main/assembler$ python3 assembler.py assembler_tests/test1.in > binfile1.txt
* ali@:~/Downloads/SAYAC_Compiler-main/assembler$
```

3. Use SAYAC ISS for running the compiled code.

[illegible]

4. In Cycle-accurate SAYAC ISS, find the execution time for calculation of the distance and angle.

همانطور که در تصویر بالاتر دیده می شود آخرین دستور اجرا شده در تصویر مشخص است (چون هر دستور در اطرافش ستاره دارد و این آخرین بخش ستاره است و در ادامه دستور دیگری اجرا نمی شود ولی پردازنده فعال است چون ران تایم زیادی به آن داده شده است) همانطور که میبینیم زمان آخرین تغییر در دستور آخر برابر 9075ns است و در نتیجه زمان اجرا چیزی حدود همان 9000ns است.

لازم به ذکر است طبق بررسی که انجام دادیم خروجی پردازنده با خروجی مورد نظر مطابقت ندارد و علت این موضوع هم مشکل در کامپایلر و اسمبلر است:

```
Initiator Bubble Comb Proc
PCregister 0000000000011000
MEM Ready before IR is:1
Ir IS: 1001001000010001
*****ADR Instruction*****
TRF read Data: TRF[1]= 0000000000000001
TRF read Data: TRF[2]= 0000000010000000
TRF Write Data: TRF[1]= 0000000010000001 Time : 13
Initiator Bubble Comb Proc
ReqProc
Target Bubble Proc
Selected target : -1
ReqProc
Target Bubble Proc
Target Bubble Proc
MEM Ready before is: 1
```

```
int main() {
    // int dx = pos_x2 - pos_x1;
    // int dy = pos_y2 - pos_y1;

    // int squared_sum = (dx * dx) + (dy * dy);
    int distance = 127;
    // distance = 0 ;
    distance = distance + 1 ;
    int max = 50;
    if (distance < max )
        distance = distance + 1 ;
    // while (distance < max ) {
    //     distance = distance + 1 ;
    // }

    // int angle_approx = 0;
    // if (dx != 0)
    //     angle_approx = dy / dx;
```

برای مثال در تصاویر بالا if نباید اجرا شه ولی اجرا میشه

به طور خلاصه به دلیل مشکل در کامپایلر چه در حوزه تبدیل اسمبلی به باینری چه سی به اسمبلی نتوانستیم کد صحیح را در مرحله اول به پردازنده بدهیم همچنین به دلیل زیاد بودن خطوط باینری (بیش از صدها خط) و نداشتن دانش کافی باینری و اسمبلی امکان تبدیل دستی آن وجود نداشت

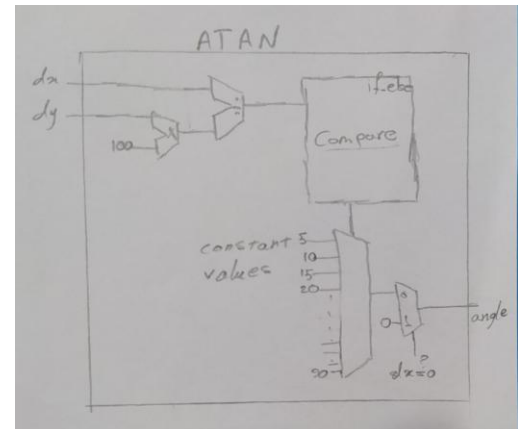
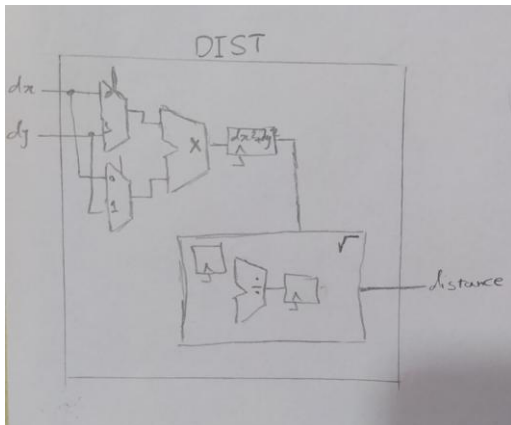
مشکل در اجرا کردن دستور های b type در کامپایلر موجود بود و در type های دیگر این مشکل مشاهده نشد.

assembler > assembler_tests > testman.in	assembler > binfile.txt
1 MSI 10 R1	1 0000000000000000
2 MSI 20 R2	2 0101000010100001
3 MSI 30 R3	3 0101000101000010
4 MSI 60 R4	4 0101110000110100
5 SUR 40 R5	5 1000010011010100
6 SUR R4 R2 R6	6 1010010000100110

مثال دیگری هم از عملکرد اشتباه اسمبلر در تصویر بالا مشخص شده است که عدد 60 را به درستی به باینری تبدیل نمی کند (این کد مربوط به سوال دوم هست که بعد از برخورد با چنین موردی آن را کاملاً به صورت دستی به کد باینری تبدیل کردیم ولی در سوال اول به خاطر اینکه بسیار پیچیده تر و همچنین طولانی تر بود امکان تبدیل دستی آن به مد باینری وجود نداشت).

Q2.

- Design two hardware units for distance and angle computation and show their block diagrams. Details of hardware are not necessary.



برای پیاده سازی بخش محاسبه فاصله به محاسبات زیر نیاز داریم:

- 1) دو ضرب: برای محاسبه ی dx^2, dy^2
- 2) یک جمع: برای جمع دو عدد بالا
- 3) محاسبه ی جذر عدد بالا: برای انجام این کار باید به تعداد مورد نیاز بسته به عددمان از عملیات تقسیم و جمع استفاده کنیم.

برای پیاده سازی بخش زاویه هم به محاسبات زیر نیاز داریم:

- 1) یک ضرب
- 2) یک تقسیم
- 3) بخش مقایسه که باید عدد حاصل از تقسیم بالا را با اعداد مختلفی مقایسه کنیم و براساس آن یک سیگنال به یک ملوٹی پلکسر ارسال کنیم تا زاویه متناسب با نتیجه مقایسه را خروجی بدهد.

2. Based on your hardware, find the approximate time in terms of clock cycles.

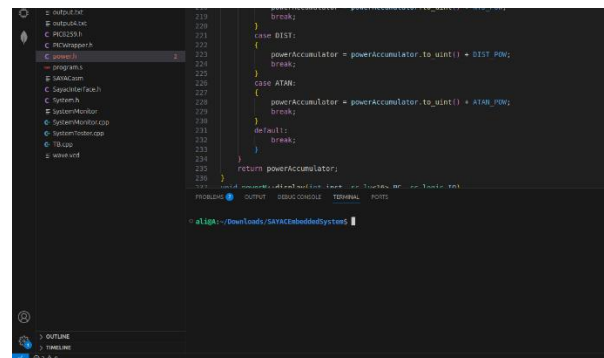
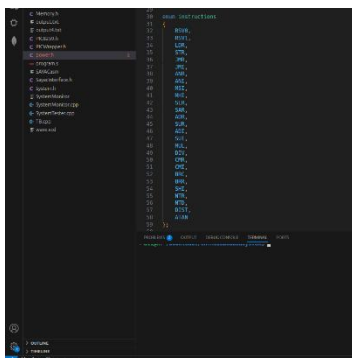
بر اساس کد های پردازنده برای عملیات ضرب و تقسیم تعداد سیکل ها مورد نیاز بر اساس متغیر $cyclesMDU$ مشخص می شود که ما هم برای محاسباتمان از همین استفاده می کنیم و طبق توضیحات بالا و کد های پردازنده تعداد سیکل مورد نیاز برای هر کدام از دو دستور بالا به این صورت می شود (برای محاسبه فاصله چون بسته به عدد ورودی ممکن است محاسبه جذر طولانی شود به مقدار N وابسته است و این وابستگی داخل کد به درستی هندل شده است) (با قرار دادن آن داخل حلقه محاسبه جذر):

$$N_{DIST} = (\sqrt{N} + 1)(12 + cyclesMDU)$$

$$N_{ATAN} = 2 \times cyclesMDU + 12$$

3. Add two instructions to SAYAC ISA for distance and angel computing with above timing.

دو مورد از فایل های پردازنده را باید تغییر میدادیم که به این صورت بود:
Power.h: برای این فایل تغییرات زیر برای اضافه کردن دو دستور جدید نیاز بود:



در IssPowerV2.h این فایل باید برای هر دستور یک opcode در نظر بگیریم و با توجه به اینکه opcode مربوط به چنین دستوراتی 4 بیتی است و در بخش enum این فایل از قبل تمام 16 حالت برای دستورات قبلی رزرو شده است نُس مجبور هستیم با توجه به کد برنامه دو مورد از دستوراتی که از آنها استفاده نکرده ایم را حذف و این دو دستور جدید را جایگزین آنها کنیم که به این صورت عمل کردیم:

همچنین تعدادی سیم جدید به صورت زیر برای محاسبات این دو بخش به ثردازنده اضافه می کنیم:

```
template <int N, int regFileAdrBit, int opcodeBit, int cyclesMDU>
void sayacInstruction<N, regFileAdrBit, opcodeBit, cyclesMDU>::abstractSimulation()
{
    Click to collapse the range.
    sc_lv<N> complement;
    sc_lv<N> templWrite;
    sc_lv<2> IR2;
    sc_lv<2> IR15;
    sc_lv<4> opcode;
    sc_lv<4> rdOrrs1;
    sc_lv<4> rs1ADR;
    sc_lv<4> rs2ADR;
    sc_lv<N> tempPC;
    sc_lv<N> signEximm;
    sc_lv<8> imm;
    sc_lv<N> Quo, Rem;
    // sc_lv<8> flags;
    sc_lv<N> dist_dx, dist_dy, dist_result;
    sc_lv<N> atan_result;
    sc_uint<N> isqrt_x, isqrt_x1, isqrt_n;
    sc_uint<N> atan_ratio;
    sc_lv<3> RFI;
    sc_logic IR8, IR9;
    PCRegister = 0x0000;

    while (true)
    {
        enum opCodes
        {
            ORSV0,
            ORSV1,
            OIns2,
            OIANR,
            OIANI,
            OIMSI,
            OIMHI,
            OISLR,
            OISAR,
            OIADR,
            OISUR,
            OIADI,
            OISUI,
            // OIMUL,
            // OIDIV,
            OODIST,
            OQATAN,
            OIns15
        };
    }
}
```

در ادامه داخل switch case ثردازنده دو دستوری را که از enum حذف کردیم را هم از آنجا حذف میکنیم(البته ما صرفا اونا رو کامنتشون کردیم) و کد های زیر رو که برای دو دستور جدید هست اضافه می کنیم:

```
case OODIST:
{
    cout << "*****DIST
Instruction*****" << endl;

    TRF_wrReg = SC_LOGIC_0;
    readRegTRF(rs1ADR, rs2ADR);

    int dx = TRF_RdData1.to_int();
    int dy = TRF_RdData2.to_int();

    unsigned int n = dx * dx + dy * dy;

    unsigned int root;

    if (n == 0)
    {
        root = 0;
    }
    else
    {

```

```
        unsigned int x = n;
        unsigned int x1 = (x + 1) / 2;

        while (x1 < x)
        {
            x = x1;
            x1 = (x + n / x) / 2;
            for (int i = 0; i < 2 *
cyclesMDU + 12; i++)
                wait();
        }

        root = x;
    }

    sc_lv<N> dist_result;
    dist_result = root;

    TRF_wrReg = SC_LOGIC_1;
    writeRegTRF(rdOrrs1, dist_result);
}
```

```

        PCregister = PCregister.to_uint() +
1;

        wait();
        break;
    }
case OATAN:
    {
        cout << "*****ATAN
Instruction*****" << endl;
        TRF_wrReg = SC_LOGIC_0;
        readRegTRF(rs1ADR, rs2ADR);
        dist_dx = TRF_RdData1;
        dist_dy = TRF_RdData2;

        sc_int<N> sdx = dist_dx.to_int();
        sc_int<N> sdy = dist_dy.to_int();

        for (int i = 0; i < 2 * cyclesMDU +
12; i++)
            wait();

            if (sdx == 0)
            {
                atan_result = (sdy >= 0) ? 90 : -90;
            }
            else
            {
                sc_int<N> ratio = (sdy * 100) /
sdx;

                sc_int<N> abs_ratio;

                if (ratio < 0)
                    abs_ratio = -ratio;
                else
                    abs_ratio = ratio;

                sc_int<N> base_angle;
                if (abs_ratio <= 8)
                    base_angle = 5;
                else if (abs_ratio <= 17)
                    base_angle = 10;
                else if (abs_ratio <= 27)
                    base_angle = 15;

```

```

                else if (abs_ratio <= 36)
                    base_angle = 20;
                else if (abs_ratio <= 47)
                    base_angle = 25;
                else if (abs_ratio <= 58)
                    base_angle = 30;
                else if (abs_ratio <= 70)
                    base_angle = 35;
                else if (abs_ratio <= 84)
                    base_angle = 40;
                else if (abs_ratio <= 100)
                    base_angle = 45;
                else if (abs_ratio <= 119)
                    base_angle = 50;
                else if (abs_ratio <= 143)
                    base_angle = 55;
                else if (abs_ratio <= 173)
                    base_angle = 60;
                else if (abs_ratio <= 214)
                    base_angle = 65;
                else if (abs_ratio <= 275)
                    base_angle = 70;
                else if (abs_ratio <= 373)
                    base_angle = 75;
                else if (abs_ratio <= 567)
                    base_angle = 80;
                else
                    base_angle = 85;

                if (sdx < 0)
                    base_angle = 180 - base_angle;
                if (ratio < 0)
                    base_angle = -base_angle;

                atan_result = base_angle;
            }

            TRF_wrReg = SC_LOGIC_1;
            writeRegTRF(rdOrrs1, atan_result);
            PCregister = PCregister.to_uint() +
1;

            wait();
            break;
        }

```

برای این بخش دیگر کد سی ننوشتیم و مستقیم به صورت دستی ابتدا کد کامپایل شده را نوشتیم که به این صورت است:

و سپس آن را به صورت دستی طبق توضیحات موجود در اسلاید های درس به کد باینری و قابل اجرا توست نردازنده تبدیل میکنیم:

و آن را شبیه سازی میکنیم که نتیجه به این صورت می شود:

همانطور که میبینیم آخرین تغییر اعمال شده در دستور آخر (یعنی دستور ATAN در این برنامه) در زمان 1165ns انجام شده است پس زمان مورد نیاز برای محاسبه فاصله و زاویه توسط پردازنده جدید چیزی حدود 1200ns است.

Q3.

1. Compare the execution time.

طبق نتایج بخش اول و دوم مدت زمان اجرای هر کدام به ترتیب تقریباً برابر 9000ns و 1200ns شد (لازم به ذکر است با توجه به اینکه کامپایلر و اسمبلر پردازنده به درستی عمل نمی کردند میتوان حدس زد که ممکن است برخی از دستورات ایجاد شده توسط آنها اضافی یا حتی غلط باشد و زمان مورد نیاز برای حالت اول کمتر باشد ولی به هر حال اختلاف بین این دو حالت کاملاً واضح است و حالت دوم که پردازنده به صورت اختصاصی میتواند محاسبات خواسته شده را انجام دهد از نظر زمانی به شدت سریعتر و بهینه تر است.

2. Describe the pros and cons of using custom instructions. (Don't forget: there is no free lunch.)

از دیدگاه ها و موارد مختلف این دو حالت را با هم مقایسه میکنیم:

سرعت و زمان اجرا (execution time): همانطور که در بخش قبل این سوال دیدیم حالت دوم به شدت سریع تر از حالت اول است و همانطور که گفته شده این موضوع کاملاً منطقی است زیرا پردازنده دوم برای انجام این عملیات طراحی شده (بخش سخت افزاری جدا برای انجام این کار دارد)، و به همین خاطر سریعتر است.

تاخیر (latency): تاخیر پردازنده دوم کمتر است زیرا سریعتر محاسبات را انجام می دهد و خروجی مورد نظر را برای ما فراهم میکند به خاطر همین آن پهنای با تاخیر کمتر و سریعتر می تواند کوتاه ترین مسیر را برای رسیدن به مقصد خود پیدا کند و به عبارتی سریعتر تصمیم بگیرد.

فضای مورد نیاز (area): با توجه به اینکه در پردازنده دوم بخش سخت افزاری مجزا برای محاسباتمان اضافه کردیم (و به نسبت هم مقدار سخت افزار استفاده شده زیاد بود: جمع کننده، صرف کننده، تقسیم کننده و موارد دیگر) در نتیجه برای ساخت چنین پردازنده ای به برد بزرگتر و فضای بیشتری نیاز داریم و اگر برای مثال پهنای نوع مینیاتوری باشد (برای مثال پهنای به اندازه کف دست) موضوع area مشکل بزرگی خواهد بود.

هزینه (cost): از نظر هزینه پردازنده دوم با اختلاف گران تر خواهد بود چون موارد اضافه شده به آن هم موارد زیادی بودند و هم موارد گران قیمت (البته که ما به بهینه ترین حالت ممکن پیاده سازی نکردیم ولی با این حال اضافه کردن چنین بخش هایی حتی در بهینه ترین حالت ممکن هم بسیار هزینه بر است).

پاور (power): پردازنده دوم از نظر مصرف برق و انرژی مورد نیاز بهینه تر است زیرا نیاز نیست تعداد زیادی مراحل میادی و محاسبات غیر مستقیم (مانند پردازنده اول) انجام داد و فاصله و زاویه را مستقیم محاسبه میکند و در نتیجه دستورات کمتری باید اجرا شود و انرژی کمتری نیاز دارد (در دستورات دیگر هم که دو پردازنده دقیقاً مشابه هم هستند)

نتیجه گیری: در آخر همانطور که در صورت سوال هم گفته شده بود there's no free lunch و بسته به کاربری پردازنده و اینکه آیا سرعت تصمیم گیری مهم است و مشکلی با هزینه اضافه و محیط بزرگتر برای پردازنده نداریم می توان این بخش ها را اضافه کرد.